CS 362 – Computer Design

Course Project – Spring 2026

Milestone 7 – Updated Design Document

Project Name: CDRLC Study Room Availability Monitor System

Team Members

Name	NetID	Email
Delon Bui	dbui9	dbui9@uic.edu
Sean Kim	skim6497	skim6497@uic.edu
Chao Liu	cliu1051	cliu1051@uic.edu
Andrew Mikielski	amiki	amiki@uic.edu

Abstract

The CDRLC Study Room Availability Monitor System is a distributed embedded network using four Arduinos to visualize real-time study room availability. Replacing tedious web portals with an intuitive physical light board, the system uses I2C communication where a central master node fetches time via WiFi and synchronizes state across three slave nodes. It features a responsive UI via LCD and buttons, automated power-saving through ultrasonic presence detection, and adaptive LED brightness using photoresistors. This project aims to streamline the student experience by dynamically parsing schedules and mapping them to hardware indicators in real-time.

Section 1: Overall Description of Project Idea

The CDRLC Study Room Availability Monitor System is a distributed embedded system designed to visualize the real-time availability of study rooms on the second floor of the Computing, Design, Research, and Learning Center (CDRLC) at UIC. The motivation for this project arose from firsthand frustration with the current experience: students must either physically walk to each room to check availability, or navigate a slow and unintuitive online portal. Neither option is efficient or well-suited to the fast-paced environment of a busy academic building.

Our solution provides an immediate, user-friendly visual map using a physical light board installed in the common area of the second floor. The board displays a to-scale layout of all study rooms, with each room represented by a bi-color LED. A green LED signals that the room is currently available, while a red LED indicates it is occupied. The system dynamically determines availability by cross-referencing a stored room reservation schedule with the current real-world time fetched from the internet via an NTP server.

As of Milestone 7, the team has made substantial hardware and software progress. The I2C master-slave communication between multiple Arduinos has been successfully prototyped on breadboards. The NTP time-fetching module is operational, and the scheduling algorithm that maps time blocks to room statuses has been coded and partially tested. The physical LED grid layout has been mapped out and wiring is underway. The team is currently in the integration phase, connecting individual subsystems into a cohesive whole.

Section 2: Project Design – Multiple Arduino Usage

The system architecture distributes processing and I/O responsibilities across four Arduino microcontrollers, each with a well-defined role in the overall network. This modular approach was chosen to avoid overloading a single controller and to allow parallel development.

Arduino 1 – Master Node (Central Server)

Arduino 1 serves as the brain of the system. It hosts an ESP8266 WiFi module (connected via SoftwareSerial or directly if using an ESP32) to periodically fetch the current UTC time from an NTP server. Using this time, it runs the core scheduling algorithm: a function that iterates through a hardcoded schedule data structure (representing reserved time blocks per room) and computes a bitmask representing the current status of each room. This status bitmask is then broadcast to the three slave Arduinos via the I2C bus. Arduino 1 also drives a piezo buzzer, which is triggered whenever the computed status changes to show a newly available room.

Arduino 2 – UI Node

Arduino 2 manages the interactive user interface. It is connected to a 16x2 LCD display (via I2C backpack to minimize pin usage), three tactile push buttons, and a 10k potentiometer. The LCD displays the current time, date, and a rolling count of open rooms. The push buttons allow users to cycle through display modes (e.g., current availability vs. next-hour forecast). The potentiometer controls LCD contrast for readability in different lighting conditions. Arduino 2 receives summary status data from the Master Node over I2C.

Arduino 3 – Display Node 1 (Wing A LEDs)

Arduino 3 controls the LED array for the left wing (Wing A) of the CDRLC floor map. It receives a room status byte from the Master Node over I2C and uses bit manipulation to set the appropriate digital output pins HIGH or LOW, controlling each room's red and green LEDs. It also integrates a photoresistor (LDR) to measure ambient light and uses analogWrite() with PWM to automatically dim the LEDs in darker conditions.

Arduino 4 – Display Node 2 (Wing B LEDs)

Arduino 4 mirrors the structure of Arduino 3 but manages the right wing (Wing B) of the floor plan. In addition to its LED array, it integrates an HC-SR04 ultrasonic distance sensor. When no student is detected within a set threshold distance (e.g., 1.5 meters), the system enters a low-power sleep mode, dimming all LEDs significantly to save energy. When proximity is detected, the display returns to full brightness immediately.

Section 3: Communication Plan Between Arduinos

All four Arduinos communicate over the Inter-Integrated Circuit (I2C) synchronous serial protocol. I2C was chosen for this project because it requires only two shared wires—SDA (Serial Data) and SCL (Serial Clock)—to connect all four nodes, which greatly simplifies physical wiring on the light board. The Wire.h library, included in the Arduino IDE, provides the software layer for I2C communication.

Arduino 1 is configured as the I2C Master by calling Wire.begin() without an address argument. Arduinos 2, 3, and 4 are configured as I2C Slaves by calling Wire.begin(address) with addresses 0x08, 0x09, and 0x0A, respectively. Each slave registers an onReceive() callback function that is invoked automatically by the hardware when data is received.

The Master transmits data in a structured 3-byte packet every 5 seconds (or immediately upon a status change). Byte 0 is a control/type byte indicating the packet type (e.g., 0x01 for room status, 0x02 for time sync). Byte 1 is a bitmask representing up to 8 rooms. Byte 2 is a simple checksum (XOR of Bytes 0 and 1) used by slaves to validate packet integrity. If the checksum fails, the slave discards the packet and waits for the next transmission cycle. This custom packet structure is a key original engineering contribution of this project.

Pull-up resistors of 4.7kΩ are placed on both the SDA and SCL lines to satisfy I2C electrical specifications. The physical I2C bus is implemented as a shared two-wire connection running along the back of the display board, with short branch wires connecting to each Arduino's A4 (SDA) and A5 (SCL) pins.

Section 4: Expected Inputs and Outputs

The system integrates eight distinct external device types across the four Arduinos, meeting the project requirement of two devices per group member. The following table summarizes all inputs and outputs:

Device	Type	Arduino Node	Function
Red/Green Bi-color LEDs (30+)	Output	Arduino 3 & 4	Visually indicate room availability on physical board
16x2 LCD (I2C backpack)	Output	Arduino 2	Show time, date, and open room count
Piezo Buzzer	Output	Arduino 1	Audible chime when a room becomes free
Tactile Push Buttons (x3)	Input	Arduino 2	Cycle LCD display modes / toggle view
10kΩ Potentiometer	Input	Arduino 2	Adjust LCD contrast for readability
Photoresistor (LDR)	Input	Arduino 3	Measure ambient light; auto-dim LEDs
HC-SR04 Ultrasonic Sensor	Input	Arduino 4	Detect student presence; trigger wake/sleep mode
ESP8266/ESP32 WiFi Module	Input/Comm	Arduino 1	Fetch real-time clock from NTP server via WiFi

As of this milestone, the LEDs, LCD, push buttons, potentiometer, and photoresistor have all been wired and individually tested. The ultrasonic sensor integration is complete and the proximity threshold logic has been validated. The WiFi/NTP module is functional and reliably returns accurate time.

Section 5: Description of Original Work

The original work in this project goes well beyond connecting commercially available sensors to an Arduino. There are three primary pillars of original engineering contribution.

First, the custom dynamic scheduling algorithm is the intellectual core of the project. Rather than hard-coding simple LED on/off states, the team has written a C++ algorithm that represents the CDRLC room reservation schedule as a structured array of time-block objects. Each object contains a room ID, a start time (in minutes from midnight), and an end time. The algorithm fetches the current NTP time, converts it to minutes from midnight, and iterates through the schedule array to set each room's status bit in a byte-sized bitmask. This bitmask is what gets transmitted over I2C. The algorithm handles edge cases such as midnight crossover, rooms booked by the hour with back-to-back reservations, and rooms that are never booked on a given day.

Second, the custom I2C packet protocol represents a meaningful systems-level engineering task. Simply using Wire.write() to send a single byte is not robust. Our three-byte packet structure (control byte + data byte + XOR checksum) adds reliability. Slave nodes validate the checksum before acting on received data, making the system resilient to I2C bus noise or incomplete transmissions—a real-world challenge with shared-bus protocols.

Third, the sensor fusion logic for power management is original and non-trivial. The system combines real-time data from both the photoresistor (continuous analog brightness reading) and the ultrasonic sensor (binary presence detection) to compute a composite LED brightness level via a priority-weighted formula. If no user is present, the system enters a deep-dim sleep mode regardless of ambient light. If a user is present, ambient light reading determines brightness via a calibrated map() call. This behavior is unique to our implementation and is not derived from any tutorial or existing project.

Section 6: How to Build the Project

Materials & Tools Required

Before beginning assembly, gather all components listed in the Materials section at the end of this document. You will also need a soldering iron, solder, wire strippers, a multimeter for continuity testing, a laptop with the Arduino IDE (version 2.x or later) installed, and craft materials (foam board or thin plywood) for the display backing.

Phase 1: Subsystem Breadboarding and Individual Testing

Begin by building and testing each Arduino node independently on separate breadboards before combining them. This is critical for isolating bugs.

- Master Node (Arduino 1): Wire the ESP8266 WiFi module to Arduino 1 using SoftwareSerial on pins D2 (RX) and D3 (TX). Upload the NTP test sketch (see Code section) and verify that it prints the correct UTC time to the Serial Monitor. Wire the piezo buzzer to pin D8 through a 100Ω resistor. Test the buzzer with a simple tone() call.
- UI Node (Arduino 2): Wire the 16x2 LCD with I2C backpack to pins A4 (SDA) and A5 (SCL). Wire three push buttons to pins D4, D5, and D6 with 10kΩ pull-down resistors. Wire the potentiometer output to pin A0. Upload the LCD test sketch and confirm text displays correctly. Test button presses using Serial.println() output.
- Display Node 1 (Arduino 3): Wire a sample set of 4 bi-color LEDs (two rooms) to digital pins D2–D5. Wire the photoresistor in a voltage divider with a 10kΩ resistor to pin A0. Upload the LED test sketch and verify correct pin-to-LED mapping. Test LDR readings using Serial.println() and verify the analogWrite() dimming logic.
- Display Node 2 (Arduino 4): Wire the HC-SR04 ultrasonic sensor with Trig to D9 and Echo to D10. Wire a sample set of 4 bi-color LEDs. Upload the ultrasonic test sketch and verify distance readings in the Serial Monitor. Test that the presence threshold logic correctly identifies a person standing within 1.5m.

Phase 2: I2C Network Integration

Once all subsystems pass individual tests, connect all four Arduinos on a shared I2C bus. Connect A4 (SDA) and A5 (SCL) of all four Arduinos together on a shared bus line. Add 4.7kΩ pull-up resistors from both SDA and SCL to the +5V rail—only one set of pull-ups is needed for the entire bus. Power all Arduinos from their USB connections during breadboarding. Upload the combined Master sketch to Arduino 1 and the appropriate Slave sketch to each remaining Arduino. Use the Serial Monitor on the Master to confirm that I2C transmissions complete without errors and that slaves are printing received data correctly to their own Serial monitors (using separate USB connections during testing).

Phase 3: Physical Board Assembly

Using foam board or thin plywood, cut a panel to represent the second-floor layout of the CDRLC. Mark the position of each study room using a scaled printout of the floor plan as a template. Drill small holes at each room's marked position and insert the bi-color LEDs from the

front, securing them with a small dab of hot glue. Route wiring through channels cut in the back of the board. Mount each Arduino to the back of the board using standoffs and M3 screws. Group wiring into labeled bundles (I2C bus, power bus, LED control lines) for cable management. Once all components are mounted, re-test the system using the breadboarded connections before soldering the final permanent connections on protoboard.

Phase 4: Final Integration and Calibration

With all components physically mounted, upload all final firmware sketches. Calibrate the photoresistor brightness curve by recording analogRead() values in a lit and a dark room, then adjusting the map() parameters accordingly. Calibrate the ultrasonic sensor threshold distance by testing with a real person standing at the intended installation location. Verify that the scheduling algorithm correctly transitions all room LEDs at a known scheduled time by temporarily hard-coding a test time into the sketch. Perform a full end-to-end test: verify NTP fetch, correct status computation, correct I2C transmission, correct LED states on the board, correct LCD display, and correct buzzer trigger.

Section 7: How to Use the Project

Overview

The CDRLC Study Room Availability Monitor is a wall-mounted, always-on display intended for students visiting the second floor of the CDRLC building. No login or smartphone is needed. Students simply look at the board.

Reading the LED Board

The main display panel shows a map of the CDRLC second floor. Each study room on the map has a bi-color LED indicator:

- Green LED (solid): The room is currently available and can be entered immediately.
- Red LED (solid): The room is currently reserved or occupied.
- All LEDs dimmed: The board is in power-saving sleep mode. Step closer to wake it up; the ultrasonic sensor will detect your presence and restore full brightness within 1 second.

Using the LCD and Buttons

The LCD panel is mounted next to the light board. It shows the current time and date at all times, as well as a live count of how many rooms are currently open. Three buttons below the LCD panel provide additional functionality:

- Button 1 (Mode): Cycles through display modes. Mode 1 shows the current open room count. Mode 2 shows the time until the next room becomes available. Mode 3 shows the number of rooms open in the next hour.

- Button 2 (Next/Previous): Steps through individual room details on the LCD when a detail view is active.
- Button 3 (Back): Returns to the default summary display.

Understanding the Audible Alert

The system emits a short two-tone chime from a small speaker when a previously fully-booked floor suddenly has a room open up. This is intended to alert students who may be waiting nearby. The chime lasts approximately 1 second and is not repeated until the next status change event.

Power and Maintenance Notes

The system is intended to run continuously while the building is open and does not require any user intervention. If the display board appears to be showing incorrect information or all LEDs are the same color, a system administrator can restart the Master Arduino by pressing its physical reset button. The board will re-fetch the current time from the NTP server and re-synchronize all nodes within approximately 10 seconds of restart.

Supporting Materials

Timeline – Work Completed (Mar 2 – Apr 3, 2026)

Week	Completed Work
Week 1 (Mar 2–8)	Finalized component list, ordered hardware, created GitHub repository, distributed development roles. Researched I2C library usage and NTP client libraries.
Week 2 (Mar 9–15)	Breadboarded Master-Slave I2C communication between Arduino 1 and Arduino 2. Verified message receipt with Serial Monitor. Implemented NTP WiFi time-fetching on Arduino 1 using the NTPClient library; confirmed accurate time retrieval.
Week 3 (Mar 16–22)	Design Presentation completed on 3/16. Wired and tested 16x2 LCD with I2C backpack on Arduino 2. Integrated push buttons with pull-down resistors; tested all three button inputs. Verified potentiometer-to-LCD contrast wiring.
Week 4 (Mar 23–29)	Developed dynamic scheduling C++ algorithm on Arduino 1. Created schedule data structure representing CDRLC room reservation blocks. Validated algorithm output against manually computed expected values. Mapped out complete LED pin assignments for both Display Nodes.
Week 5 (Mar 30–Apr 3)	Integrated photoresistor with analogRead() and PWM dimming on Arduino 3. Integrated HC-SR04 ultrasonic sensor on Arduino 4 and tested proximity threshold logic. Designed 3-byte I2C packet structure and implemented XOR checksum validation on all slave nodes.

Timeline – Work Remaining (Apr 6 – Apr 29, 2026)

Week	Planned Work
Week 6 (Apr 6–12)	Full I2C network integration: connect all four Arduinos on shared bus with pull-up resistors. Run end-to-end test: NTP fetch → schedule computation → I2C broadcast → LED state update → LCD update. Debug any timing or synchronization issues.
Week 7 (Apr 13–19)	Begin physical board assembly. Cut and prepare foam board panel. Drill LED holes following floor plan template. Mount LEDs and route wiring. Mount Arduinos on back with standoffs. Complete protoboard soldering for permanent connections.
Week 8 (Apr 20–26)	Full system integration testing on assembled board. Calibrate LDR brightness curve and ultrasonic threshold. Test edge cases: midnight crossover, back-to-back reservations, WiFi dropout handling. Implement buzzer chime logic for room-freed events. Final aesthetic polish.
Week 9 – Demo (Apr 27)	Monday 4/27/26: Project Demonstration during scheduled lab session. Present fully assembled and operational system to instructor and class.
Week 9 – Final Doc (Apr 29)	Wednesday 4/29/26: Final Design Document (Milestone 9) submitted to Gradescope as a PDF with all implementation details, final Fritzing diagrams, complete code listings, and post-project reflection.

Updated List of Materials

#	Component	Qty	Notes
1	Arduino Uno R3 or Mega 2560	4	One per team member
2	ESP8266 WiFi Module (or ESP32)	1	Attached to Master Node
3	Bi-color Red/Green LEDs (or separate Red + Green)	30+	Two per study room
4	16x2 Liquid Crystal Display with I2C Backpack	1	PCF8574 I2C expander
5	HC-SR04 Ultrasonic Distance Sensor	1	On Display Node 2
6	Photoresistor (LDR)	1	On Display Node 1
7	Tactile Push Buttons	3	On UI Node
8	10kΩ Potentiometer	1	LCD contrast control
9	Piezo Buzzer	1	On Master Node

#	Component	Qty	Notes
10	Resistors: 100Ω, 220Ω, 330Ω, 4.7kΩ, 10kΩ	Assorted	LED current limiting, pull-ups, pull-downs, LDR divider
11	Breadboards and Jumper Wires	4 sets	For prototyping phase
12	Protoboards / Perfboards	4	For permanent assembly
13	Foam board or thin plywood (24" x 18")	1	Physical display backing
14	USB cables, standoffs, M3 screws, hot glue	As needed	Assembly and mounting

Updated List of References

1. Arduino Official Reference. "Wire Library (I2C)." Arduino Documentation. https://www.arduino.cc/en/reference/wire
2. Fabrice Weinberg. "NTPClient Library for Arduino." GitHub. https://github.com/arduino-libraries/NTPClient
3. Frank de Brabander. "LiquidCrystal_I2C Library." Arduino Reference. https://www.arduino.cc/reference/en/libraries/liquidcrystal-i2c/
4. "HC-SR04 Ultrasonic Sensor Interfacing with Arduino." Arduino Project Hub. https://projecthub.arduino.cc/Isaac100/getting-started-with-the-hc-sr04-ultrasonic-sensor-7cabe1
5. "Photoresistor Basics – Using an LDR with Arduino." Adafruit Learning System. https://learn.adafruit.com/photocells/using-a-photocell
6. "ESP8266 Arduino Core Documentation." ESP8266 Community Forum. https://arduino-esp8266.readthedocs.io/en/latest/
7. Khaled Magdy. "Arduino I2C Tutorial – Master and Slave Communication." DeepBlueMbedded. https://deepbluembedded.com/arduino-i2c-tutorial/
8. "TinkerCad Circuits – Online Arduino Simulator." Autodesk. https://www.tinkercad.com/circuits

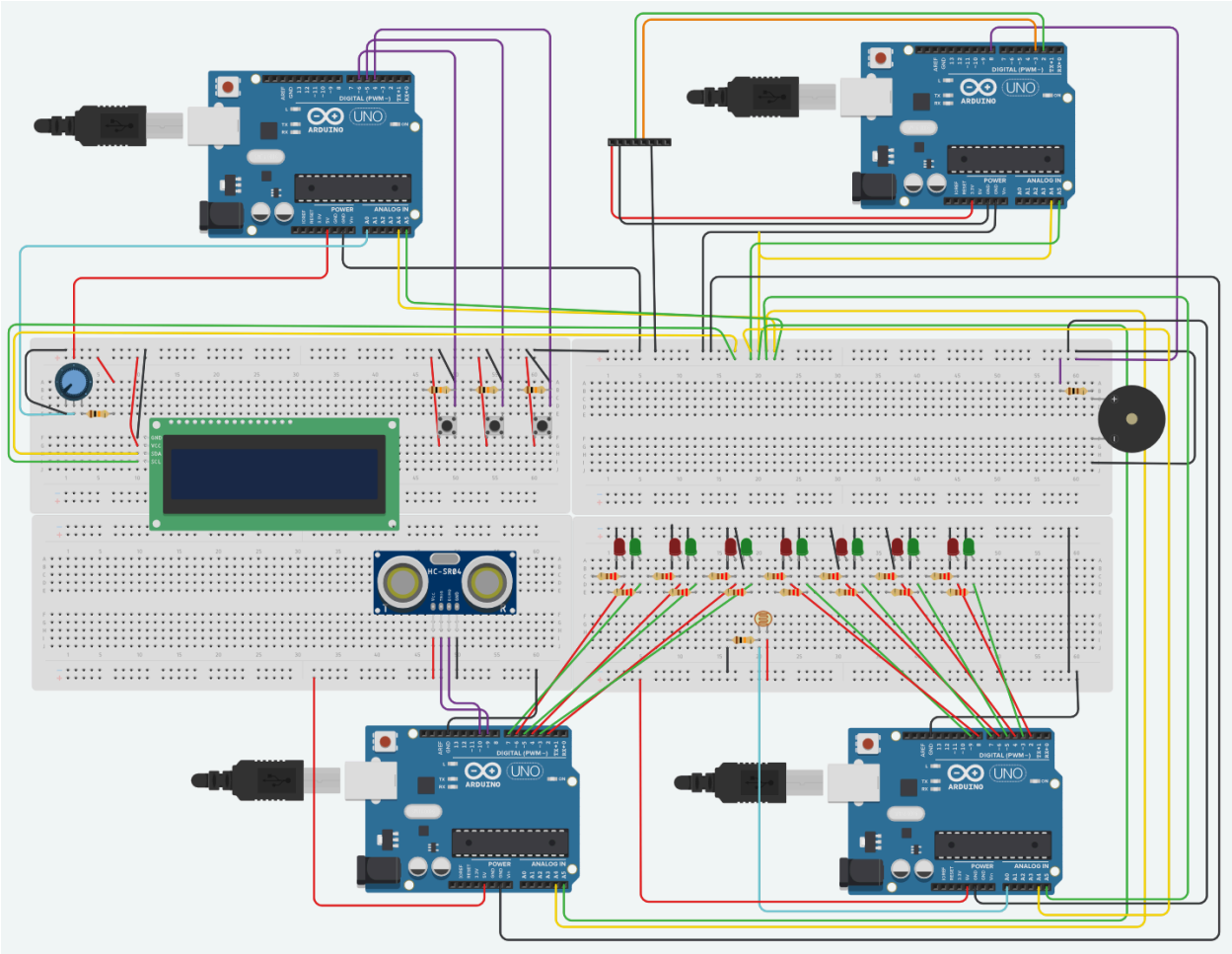
Hardware Diagrams

The team is using TinkerCad Circuits to design and simulate the hardware schematics for each Arduino node. TinkerCad provides Fritzing-equivalent visual circuit diagrams that show all wiring, component placement, and pin connections. The diagrams below correspond to each subsystem and will be replaced with full Fritzing exports in the Milestone 9 Final Design Document.

The Wifi module was not available in TinkerCad so it is loosely represented with an 8-pin header.

Master Node: Top-Right
UI Node: Top-Left

Display Node 2 (Wing B): Bottom-Left
Display Node 1 (Wing A): Bottom-Right



Updated Code Sketches

Master Node – Arduino 1 (Excerpt: NTP Fetch + Schedule Algorithm + I2C Broadcast)

```
#include <Wire.h>
#include <ESP8266WiFi.h>
#include <NTPClient.h>
#include <WiFiUdp.h>

const char* ssid     = "UIC-WiFi";
const char* password = "";  // Open network

#define SLAVE_UI_ADDR    0x08
#define SLAVE_DISP1_ADDR 0x09
#define SLAVE_DISP2_ADDR 0x0A
#define BUZZER_PIN       8

WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", -21600, 60000); // UTC-6 (Chicago)

// Schedule: { roomID, startMinutes, endMinutes }
struct TimeBlock { uint8_t roomID; int startMin; int endMin; };

TimeBlock schedule[] = {
  {0, 480, 600},  // Room 201: 8:00-10:00 AM
  {1, 540, 660},  // Room 202: 9:00-11:00 AM
  {2, 600, 720},  // Room 203: 10:00 AM-12:00 PM
  {3, 660, 780},  // Room 204: 11:00 AM-1:00 PM
  // ... (full schedule array)
};
const int SCHEDULE_LEN = sizeof(schedule) / sizeof(schedule[0]);

uint8_t prevStatus1 = 0xFF;  // Track previous state for buzzer trigger
uint8_t prevStatus2 = 0xFF;

void setup() {
  Wire.begin();           // Join I2C bus as Master
  Serial.begin(9600);
  pinMode(BUZZER_PIN, OUTPUT);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {}
  timeClient.begin();
  timeClient.update();
}

// Returns bitmask of 8 rooms: bit=0 means available, bit=1 means occupied
uint8_t calcRoomStatus(int nowMin, uint8_t startID, int count) {
  uint8_t mask = 0x00;
  for (int i = 0; i < SCHEDULE_LEN; i++) {
    if (schedule[i].roomID >= startID && schedule[i].roomID < startID + count)
{
      if (nowMin >= schedule[i].startMin && nowMin < schedule[i].endMin) {
        mask |= (1 << (schedule[i].roomID - startID));  // Mark as occupied
      }
    }
```

```
  }
  return mask;
}

void sendI2CPacket(uint8_t addr, uint8_t ctrl, uint8_t data) {
  uint8_t checksum = ctrl ^ data;  // XOR checksum
  Wire.beginTransmission(addr);
  Wire.write(ctrl);
  Wire.write(data);
  Wire.write(checksum);
  Wire.endTransmission();
}

void loop() {
  timeClient.update();
  int nowMin = timeClient.getHours() * 60 + timeClient.getMinutes();

  uint8_t status1 = calcRoomStatus(nowMin, 0, 8);  // Wing A: rooms 0-7
  uint8_t status2 = calcRoomStatus(nowMin, 8, 8);  // Wing B: rooms 8-15

  sendI2CPacket(SLAVE_DISP1_ADDR, 0x01, status1);
  sendI2CPacket(SLAVE_DISP2_ADDR, 0x01, status2);
  sendI2CPacket(SLAVE_UI_ADDR, 0x02, __builtin_popcount(~status1) +
__builtin_popcount(~status2));

  // Buzzer: trigger if a previously occupied room is now free
  if ((prevStatus1 & ~status1) || (prevStatus2 & ~status2)) {
    tone(BUZZER_PIN, 880, 200);
    tone(BUZZER_PIN, 1047, 400);
  }
  prevStatus1 = status1; prevStatus2 = status2;
}
```

Display Node – Arduino 3 (Excerpt: I2C Receive + LED Control + LDR Dimming)

```
#include <Wire.h>

#define MY_ADDRESS 0x09

// LED pin pairs: { greenPin, redPin } for each room (Wing A, 8 rooms)
const int LED_PINS[8][2] = {
  {2, 3}, {4, 5}, {6, 7}, {8, 9},
  {10, 11}, {12, 13}, {A1, A2}, {A3, A4}
};
const int LDR_PIN = A0;
volatile uint8_t roomStatus = 0x00;
volatile bool newData = false;

void setup() {
  Wire.begin(MY_ADDRESS);
  Wire.onReceive(receiveEvent);
  for (int i = 0; i < 8; i++) {
    pinMode(LED_PINS[i][0], OUTPUT);  // Green
    pinMode(LED_PINS[i][1], OUTPUT);  // Red
  }
```

```
}

void receiveEvent(int numBytes) {
  if (numBytes == 3) {
    uint8_t ctrl = Wire.read();
    uint8_t data = Wire.read();
    uint8_t chk  = Wire.read();
    if ((ctrl ^ data) == chk && ctrl == 0x01) {  // Validate checksum + type
      roomStatus = data;
      newData = true;
    }
  } else { while (Wire.available()) Wire.read(); }  // Flush bad packet
}

void updateLEDs(uint8_t status, uint8_t brightness) {
  for (int i = 0; i < 8; i++) {
    bool occupied = (status >> i) & 0x01;
    analogWrite(LED_PINS[i][1], occupied ? brightness : 0);    // Red
    analogWrite(LED_PINS[i][0], occupied ? 0 : brightness);    // Green
  }
}

void loop() {
  int ldrValue = analogRead(LDR_PIN);  // 0 (dark) to 1023 (bright)
  // Map ambient light to LED brightness: darker room = dimmer LEDs
  uint8_t brightness = map(ldrValue, 100, 900, 30, 255);
  brightness = constrain(brightness, 30, 255);

  if (newData) {
    newData = false;
    updateLEDs(roomStatus, brightness);
  }
}
```

UI Node – Arduino 2 (Excerpt: LCD Display + Button Mode Cycling)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define MY_ADDRESS 0x08
#define BTN_MODE    4
#define BTN_NEXT    5
#define BTN_BACK    6
#define POT_PIN     A0

LiquidCrystal_I2C lcd(0x27, 16, 2);
volatile uint8_t openRooms = 0;
int displayMode = 0;  // 0=Summary, 1=NextAvail, 2=NextHour
bool lastModeBtn = HIGH;

void setup() {
  Wire.begin(MY_ADDRESS);
  Wire.onReceive(receiveEvent);
  lcd.init(); lcd.backlight();
  pinMode(BTN_MODE, INPUT_PULLUP);
```

```
  pinMode(BTN_NEXT, INPUT_PULLUP);
  pinMode(BTN_BACK, INPUT_PULLUP);
}

void receiveEvent(int bytes) {
  if (bytes == 3) {
    uint8_t ctrl = Wire.read();
    uint8_t data = Wire.read(); Wire.read();  // discard checksum here
    if (ctrl == 0x02) openRooms = data;
  } else { while (Wire.available()) Wire.read(); }
}

void loop() {
  // Adjust LCD contrast via potentiometer (V0 pin of LCD wired externally)
  // (Contrast wired to wiper of pot; no code needed for hardware contrast)

  bool modeBtn = digitalRead(BTN_MODE);
  if (modeBtn == LOW && lastModeBtn == HIGH) {  // Falling edge = button press
    displayMode = (displayMode + 1) % 3;
    lcd.clear();
  }
  lastModeBtn = modeBtn;

  lcd.setCursor(0, 0);
  if (displayMode == 0) {
    lcd.print("Rooms Open: "); lcd.print(openRooms);
    lcd.setCursor(0, 1); lcd.print("Press MODE->details");
  } else if (displayMode == 1) {
    lcd.print("Next Free: ");
    lcd.setCursor(0, 1); lcd.print("--:-- [Room ###]");  // TBD: integrate
schedule
  } else {
    lcd.print("In 1hr Open:");
    lcd.setCursor(0, 1); lcd.print("-- rooms avail");
  }
}
```